

LLMs for Test Input Generation for Semantic Applications

Zafaryab Rasool, Scott Barnett, David Willie, Stefanus Kurniawan, Sherwin Balugo, Srikanth Thudumu, Mohamed Abdelrazek

{zafaryab.rasool,scott.barnett,david.willie,stefanus.kurniawan,s.balugo}@deakin.edu.au

{srikanth.thudumu,mohamed.abdelrazek}@deakin.edu.au

Applied Artificial Intelligence Institute, Deakin University
Geelong, Australia

ABSTRACT

Large language models (LLMs) enable state-of-the-art semantic capabilities to be added to software systems such as semantic search of unstructured documents and text generation. However, these models are computationally expensive. At scale, the cost of serving thousands of users increases massively affecting also user experience. To address this problem, semantic caches are used to check for answers to similar queries (that may have been phrased differently) without hitting the LLM service. Due to the nature of these semantic cache techniques that rely on query embeddings, there is a high chance of errors impacting user confidence in the system. Adopting semantic cache techniques usually requires testing the effectiveness of a semantic cache (accurate cache hits and misses) which requires a labelled test set of similar queries and responses which is often unavailable. In this paper, we present VaryGen, an approach for using LLMs for test input generation that produces similar questions from unstructured text documents. Our novel approach uses the reasoning capabilities of LLMs to 1) adapt queries to the domain, 2) synthesise subtle variations to queries, and 3) evaluate the synthesised test dataset. We evaluated our approach in the domain of a student question and answer system by qualitatively analysing 100 generated queries and result pairs, and conducting an empirical case study with an open source semantic cache. Our results show that query pairs satisfy human expectations of similarity and our generated data demonstrates failure cases of a semantic cache. Additionally, we also evaluate our approach on Qasper dataset. This work is an important first step into test input generation for semantic applications and presents considerations for practitioners when calibrating a semantic cache.

CCS CONCEPTS

• **Software and its engineering** → Empirical software validation.

KEYWORDS

Large Language Model, Query Evaluation, Question Answering, Semantic Cache, Test Input Generation

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CAIN 2024, April 14–15, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0591-5/24/04...\$15.00

<https://doi.org/10.1145/3644815.3644948>

ACM Reference Format:

Zafaryab Rasool, Scott Barnett, David Willie, Stefanus Kurniawan, Sherwin Balugo, Srikanth Thudumu, Mohamed Abdelrazek. 2024. LLMs for Test Input Generation for Semantic Applications. In *Conference on AI Engineering Software Engineering for AI (CAIN 2024)*, April 14–15, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3644815.3644948>

1 INTRODUCTION

Software development kits for creating applications that use large language models (LLMs) are starting to emerge i.e Microsoft’s Semantic Kernel¹ and Langchain². These SDKs are growing in popularity as LLMs are able to 1) resolve ambiguities in natural language queries, 2) reason over unstructured documents, and 3) synthesise a human like responses to prompts [5]. Semantic caches are becoming an important aspect of these SDKs as developers seek to create applications with a) lower latency, b) lower costs, and c) reduced environmental impact from using less compute resources (i.e. no repeat calls to LLMs with billions of parameters).

A semantic cache enables similar queries (as defined by a similarity measure rather than an exact match) to match cached responses. However, testing the robustness of a semantic cache is challenging due to a) application specific calibration to balance the cache hits/misses ratio, b) domain specific query/response pairs, and c) a suitable similarity measure that handles the nuance in natural language (see Table 1). Ensuring the robustness of an application with a semantic cache relies on a representative sample of query/response pairs for calibration and testing.

Query	<i>Do I have to reference in my Assignment 2?</i>
Cached Query	<i>In Assignment 2, how many university references should I use?</i>

Table 1: An example of an incorrect cache hit using the semantic cache GPTCache³.

LLMs have been used to generate data for software applications [4, 9–12, 14] and for testing information retrieval based applications [3]. They base their work on the assumption of pre-existing questions which for new applications is not available. Recent work has also explored the use of LLMs for generating data for training Q&A systems. However, their work focuses on the volume of questions from documents rather than subtle variations that would expose the limits of a semantic caching strategy. In summary, prior work has not considered the problem of test input generation for

¹<https://learn.microsoft.com/en-us/semantic-kernel/overview/>

²https://python.langchain.com/docs/get_started/introduction

³<https://github.com/zilliztech/GPTCache>

testing LLM based Q&A systems. Our goal is to improve the robustness of Q&A systems by test input generation and demonstrating the effectiveness of finding limitations in a semantic cache.

In this paper, we propose an innovative approach that uses LLMs for generating domain specific query/response pairs. We call this approach as Variation Generator (VaryGen). VaryGen uses LLMs to a) generate queries, b) create hard negatives, and c) verify the quality of the generated pairs. We take a two stage approach, first we synthesise domain specific questions from a collection of documents, optionally including domain specific terms in the generation process. Second, from the synthesised questions we create variations resulting in hard negatives and close matches. To evaluate our approach, we 1) conducted a qualitative analysis of the synthesised questions (N=100), and 2) through a case study of an open source semantic cache, GPTCache. Our results found limitations in the default configurations of GPTCache demonstrating that our approach is suitable for test input generation. However, the qualitative evaluation revealed additional filters are required to improve the variations of questions which we leave for future work.

The contributions arising from this work are:

- An automated approach for synthesising query/response pairs that contain hard negatives. This approach has been designed for testing applications that include QandA or a semantic cache for natural language queries.
- A qualitative analysis of the challenges with using LLMs for synthesising test input data.
- An empirical case study with an open source semantic cache, GPTCache. This showed the nuances in similarity measures that software engineers need to test to improve the robustness of their applications.

2 SEMANTIC CACHE

For the purpose of this paper, we investigate the use of a semantic cache as part of a question and answering system where the queries are questions and the answers are the responses. Semantic caching has been a long studied problem [1, 2, 7, 8, 13] for efficient retrieval of query responses by reusing already extracted data. A semantic cache is designed to optimise the performance and efficiency of systems that process large volumes of data, such as those based on LLMs. Unlike traditional caches that store data based on the exact queries or data requests, a semantic cache understands the meaning or "semantics" of the data and the queries. This allows it to not just store exact query results, but also data that is semantically related to these queries. The semantics are represented by a text embedding, an array of floating point values that has been trained with a language model. When a new query is made, the semantic cache retrieves data that is contextually relevant, even if the new query is not an exact match to previous ones.

Text embeddings are imprecise for subtle variations in natural language and for domain specific terminology. When using a semantic cache, a search algorithm such as K-nearest neighbours is used to find relevant embeddings before filtering to a single result. This approach requires calibration as it will always return a result even if the answers aren't similar. The robustness of an LLM based application requires a well calibrated semantic cache. Incorrect configuration results in incorrect cache hits (i.e. the user receives

an incorrect answer), or many incorrect cache misses - mitigating the benefits of using a semantic cache. Each application requires calibration due to domain specific terminology and diversity of content i.e. the quality of the embeddings for each domain is unknown until tested. To calibrate a semantic cache a representative dataset is required that includes both true matches and almost but incorrect matches.

Thus a test input generation approach for semantic caches requires the following:

- Generate domain relevant queries automatically.
- Include a test oracle to ensure that generated queries are relevant to the domain.
- Create semantically "close" variations of queries (hard negatives and true pairs).
- Automated to scale the data generating process to support accurate calibration.

3 VARYGEN

Based on the discussion above, we propose a test input generation approach using LLM. This 3-step approach, (detailed below) depicted in Figure 1, combines the strengths of LLMs, domain knowledge, and rigorous evaluation to generate diverse, contextually relevant, and high-quality test input data for semantic applications.

3.1 Query And Answer Synthesis

In formulating our approach, we made three assumptions: 1) a corpus of documents is available, 2) the documents are a semantic representation (i.e. an embedding vector that represents the semantic content of the document), and 3) a way to retrieve answers to queries from the documents is available.

The initial step involves extracting relevant answers from the provided document. The process of answer extraction from documents for test input generation involves utilizing LLMs and a document corpus and is as follows:

First, we provide a document to an LLM and prompt it to extract facts from the document. Most documents consist of multiple facts or sentences that can serve as an answer to specific questions. Thus, we use these facts as answers for generating questions. Note that, if the document is too long or consists of multiple documents, a retriever can be used to extract relevant facts from it. However, for simplicity, we provide full document passages in the prompt to an LLM to extract answers. We then again prompt the LLM to generate questions for each of the extracted answer. Here, proper prompting and using extracted answers restrict the generated questions to the scope of the document only, thereby obtaining domain relevant queries automatically.

3.2 Query Evaluation

The generated queries obtained in the previous step need to be verified for correctness and whether they align contextually with the content of the document, which is the aim of this step.

This step validates the accuracy of the generated queries by mapping them back to the original document. For each query, the top N documents are retrieved and assessed whether the original document is included in the returned set. The queries that extract relevant documents are included while the others are excluded from

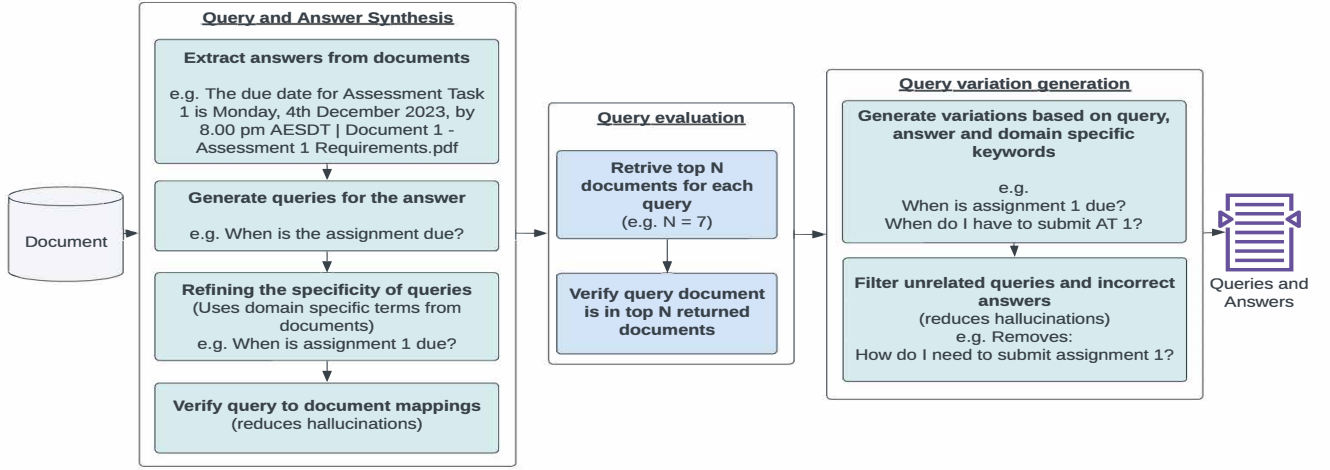


Figure 1: The 3 stages in VaryGen : 1) Query Generation, 2) Query Evaluation and 3) Query Variation Generation. Green boxes indicate steps that involve a LLM and blue boxes that are automated without the use of an LLM.

Question: {question}
Answer: {answer}

Generate variations ONLY from the provided Question above and ONLY use the Answer to constrain the generated questions.

Variation Guidelines: {...}
Response:

Table 2: Prompt template for Query Variation Generation

the set. The refined set of generated queries is then used in the next step to generate variations of the queries.

Thus, this verification step ensures that the obtained queries align contextually with the content of the document (i.e., they are domain specific), establishing a reliable connection between the generated questions and the source material.

3.3 Query Variation Generation

This step focuses on generating semantically similar variations of the queries obtained in the previous step. The idea is to generate as diverse variations of the query as possible while still maintaining semantic similarity.

We prompt the LLM to generate variations of the queries making sure that: (i) the queries are diverse i.e. different in terms of phrasing, structure, or specific keywords, and (ii) any unrelated or hallucinated queries are removed. This maintains the quality of the generated variations. We provide both the question and the answer in the prompt in order to keep the queries related to the extracted content and that the domain knowledge is reflected in the generated variations.

Utilising proper prompting, this step refines the queries by incorporating specific terms, concepts, or structures relevant to the domain, ensuring that the generated test inputs align closely with the intricacies of the subject matter. Table 2 provides an example of a prompt used to achieve the above objective. The guidelines help to provide relevance and specificity in the query variations.

Thus, using the above approach, we get the required query variations that can be used to test semantic applications. Next, we discuss the evaluation of our input data generation approach.

4 PRELIMINARY EVALUATION

In this section, we discuss the dataset details and the evaluation strategy along with the results of the evaluations.

4.1 Dataset Description

We use two datasets for our study. The Qasper dataset [6] is an information-seeking dataset consisting of research papers on natural language processing. The dataset contains question-answers with different question types such as abstractive, extractive, boolean and unanswerable. We filter the dataset by removing the unanswerable ones for our purpose.

The Assignment dataset is a manually curated dataset containing 61 questions crafted from sample questions representing what students are likely to ask during a university subject. The dataset consists of questions regarding the assignments such as word limit, requirements, and use of GenAI. As some questions can be similar, we further excluded questions that are too similar by manual inspection, leaving only 52 unique questions. This Assignment dataset is an original creation of the authors.

4.2 Qualitative Evaluation

The focus of VaryGen was to create an automated query and answer generation pipeline to support software engineers working with a semantic cache. We conducted evaluations in two phases to ensure a thorough analysis of the VaryGen approach. These evaluations covering the different steps of VaryGen ensure representative samples of each of the core stages is evaluated.

Method: The initial evaluation involved a human assessment of the generated questions. We conducted a random sampling of 100 questions, scrutinizing each to determine if the associated answers appeared relevant to the generated questions. Our criterion

Strategy	Correct Hits	Incorrect Hits	Correct Misses	Incorrect Misses
DistilBERT	1234	885	19	128
SBERT	1422	360	38	446
ONNX	1111	314	35	806

Table 3: Results from evaluating the generated dataset against three semantic caching strategies

for marking a question as ‘Correct’ included assessing both the relevance of the answer to the question and the overall realism of the question – whether it resembled the type of questions students would typically ask. Questions meeting these criteria were marked as ‘Correct’, while those that did not were labelled as ‘Incorrect’. We conducted an initial evaluation for the Assignment dataset as it consists of documents and not for the Qasper dataset, which consisted of questions that could be directly used for the next step.

The second phase of evaluation involved a human assessment of the query variations generated for each query. In this stage, we randomly selected 100 question-answer pairs and examined each question’s correctness by comparing it against the original answer. Essentially, our focus was on verifying whether the generated questions aligned sensibly with the corresponding answer. This evaluation is done for both datasets.

Results: The results of the initial evaluation on the Assignment dataset indicate that 81% of the generated questions were realistic, while only 19% did not satisfy our criteria. This demonstrates that our approach generates relevant questions that are domain-specific and realistic. For example, questions such as “What do I need to formulate that is original and related to the topic I have chosen for Assessment Task 2?” may not be asked by a student. We now discuss the evaluation results on query variations. For the Assignment dataset, we found that 80% of the sampled query variations were correct, while 20% were incorrect. Similarly, for the Qasper dataset, the correct was 84% while the incorrect % was only 16.

These results from the evaluations emphasise that most generated questions and the variations of the query for both datasets were semantically similar to the original questions from which the variations were generated.

4.3 Semantic Cache Case Study

We analysed the performance of an open source semantic cache library using the generated dataset.

Method. For the purpose of our case study, we evaluated GPTCache [5], an open source semantic cache library with various semantic caching strategies. GPTCache was selected due to a) intended use for LLM applications, and b) popular (over 5000 stars). We used the Assignment dataset and evaluated the performance of GPTCache using three strategies with a similarity threshold of 0.9 (selected through empirical evaluation):

- DistilBERT (base, uncased) embeddings with Cosine Similarity for similarity evaluation
- SBERT embeddings with GPTCache’s SBERT CrossEncoder implementation for similarity evaluation
- ONNX embeddings with GPTCache’s ALBERT ONNX implementation for similarity evaluation

The following metrics were used to measure the performance of GPTCache: Number of correct cache hits, Number of incorrect

Strategy	Query	Cached Key
DistilBERT	For the success of this unit, is the use of generative AI necessary?	Does this unit require to use generative AI?
SBERT	In Assignment 2, how many academic citations are necessary?	How many scholarly references should be included in Assessment 2?
ONNX	How extensive should my description of using generative AI be for Assessment 1?	How comprehensive should I be when explaining my use of generative AI for Assignment 1?

Table 4: True Positive - Correct Cache Hit

Strategy	Query	Cached Key
DistilBERT	What is the word limit for the background section of my Assessment 1?	What is the recommended word count for the aims section of my report for AT1?
SBERT	What is the directive for Assignment 1?	How many words should go into my campaign strategy for Assignment 1?
ONNX	How should I proceed with Assessment 2?	How can I get a high distinction in Assessment 2?

Table 5: False Positive - Incorrect Cache Hit

Strategy	Query	Expected Key
DistilBERT	What should the layout of my AT1 look like?	What sections should I have in my AT1?
SBERT	What’s the brief for Assignment 1?	What do I need to do for Assignment 1?
ONNX	What methodologies are feasible for Assignment 2?	What methods can I use for Assignment 2?

Table 6: False Negatives - Incorrect Cache Miss

cache hits, Number of correct cache misses and Number of incorrect cache misses. Correct Hits indicate a correct value was retrieved from the cache. Incorrect Hits indicate a non-matching value was returned. Correct Misses indicate no value was returned when no value was expected, and Incorrect Misses indicate a value was not returned where one was expected.

Results. We used the Assignment dataset consisting of a total of 2266 questions including the original questions and their variations generated using our approach. We put the dataset of questions into the cache, one by one, and measure the above metrics. The results of our evaluation are shown in Table 3 which demonstrates the generated question set was capable of providing examples that resulted in Correct hits, Incorrect Hits and Incorrect Misses on all three caching strategies.

We discuss some sample examples of the above results. For correct cache hits, as shown in Table 4, all three sample query, while different in structure and phrasing, are similar in meaning to the respective cached key, which validates the relevance and accuracy of the synthesized questions. This highlights the robustness and effectiveness of our test input generation approach in capturing semantic relevance across many different formulations of queries.

Regarding incorrect cache hit examples shown in Table 5, the first example shows that although both queries pertain to Assessment 1, they focus on different aspects (background vs. aims), making them semantically distinct. The reason for this incorrect cache hit could be attributed to the limitations in the semantic cache’s ability to discern subtle variations in the meaning of queries. The second and third examples are also semantically different in terms of their requirements and follow similar explanations.

Regarding incorrect cache miss examples shown in Table 6 in the first example, the semantic cache fails to identify the correct match, as it may not fully capture the specific intent (sections and layouts) behind the generated query. For the second example, the generated query is seeking information about the brief or instructions for Assignment 1, while the expected cached query is a more general inquiry about the tasks required for Assignment 1. The semantic cache fails possibly due to the subtle differences in the queries semantics. Similarly, for the third example, the terms "methodologies" and "methods" may not have a direct one-to-one correspondence in the semantic embeddings.

Thus, our approach is capable of uncovering scenarios where the semantic cache strategies may fail to identify relevant matches or may incorrectly retrieve matches. This demonstrates areas for refinement in the specificity, diversity, and coverage of the generated queries to enhance the performance of semantic caching systems.

5 RELATED WORK

Recent studies have demonstrated the potential for leveraging LLMs to automatically generate various test artifacts including test cases, test inputs, and test oracles [14]. Kang et al. [9] proposed an approach for using an LLM to reproduce bugs and generate reliable test cases to suggest to developers. Yu et al. [17] demonstrated how well LLMs can adapt to diverse systems while creating test scripts. Liu et al. [11] introduced an LLM-based approach to intelligently generate semantic input text based on GUI context. Other works have focused on using LLMs or semantic knowledge bases to generate valid test inputs. Alonso et al. [4] proposed ARTE, which uses knowledge bases such as DBpedia to extract realistic web API test inputs based on API specifications. Mariani et al. [12] introduced Link, which analyses a GUI’s input fields, queries DBpedia to find matching data, and generates complex test inputs. Wanwarang et al. [15] leveraged both static and dynamic analysis along with an LLM to generate test inputs. Some studies have investigated using LLMs to generate test oracles, which determine the correctness of test outputs. Yang et al. [16] used an LLM to create knowledge bases to guide testers on what outputs to validate. Alaofi et al. [3] showed LLMs can generate variations of queries to create test oracles.

LLMs show promise for the automated generation of various test artefacts, reducing manual effort while improving test coverage and effectiveness. However, there remain open challenges in improving the reliability and semantic correctness of LLM-generated tests.

6 LIMITATIONS AND FUTURE RESEARCH DIRECTIONS

While our approach demonstrates promise in generating domain-specific query/response pairs for testing LLMs and semantic applications, certain limitations underscore areas for future research.

The need for additional filters to enhance the quality of query variations, identified through our evaluation, prompts further investigation into advanced techniques for refinement. Additionally, our approach currently lacks human validation during data generation, which means some generated questions may be of lower quality or incorrect. However, we argue that an automated, inexpensive dataset that uncovers software limitations, as shown in our case study, still provides utility. Incorporating human annotation to improve dataset quality remains an area for future work.

Additionally, the inherent challenge of subtle semantic nuances and imprecisions in text embeddings requires attention, urging the exploration of methods to fine-tune LLMs or incorporate domain-specific embeddings. Our evaluation also focused on a single semantic cache, so further empirical comparisons of multiple semantic caching systems should be carried out, which is now feasible using VaryGen. Other potential areas for future work include performing ablation studies to reduce data generation costs for increased scale, defining quality models and metrics to improve query generation, additional empirical evaluations of open source caches, generating datasets to produce guards protecting semantic applications, incorporating diversity metrics to broaden the scope of questions, and further analysis of how synthesis techniques could be tailored and improved for diverse domains and applications.

Addressing the limitations and exploring these research directions will contribute to the continual improvement and advancement of test input generation methodologies for LLMs and semantic applications.

7 CONCLUSION

This paper has explored the evolving role of software development kits (SDKs) that leverage large language models (LLMs) to enable more advanced natural language applications. Semantic caches have emerged as a pivotal component within these SDKs, aiming to reduce latency, costs, and environmental impact by avoiding redundant calls to the LLM. However, the imprecision of text embeddings requires careful calibration of semantic caches to prevent incorrect cache hits. To address this, we have proposed and evaluated an innovative approach for automated test input generation using LLMs themselves. Through a two-stage process of synthesising domain-specific queries and creating semantically close variations, including hard negatives, representative test data can be generated at scale. Both the qualitative analysis and semantic cache case study validate the suitability of this approach for the robust testing necessary to ensure cache calibration. This research contributes a holistic perspective, not just on test input generation, but on the end-to-end robustness and calibration needs of semantic caches in this evolving landscape of LLM-based SDKs. The proposed approach paves the way for enhanced software development practices that fully harness advanced language models while safeguarding reliability.

8 ACKNOWLEDGEMENTS

To the reviewers for their valuable comments, and to Rajesh Vasa and Kon Mouzakis for their insightful guidance and unwavering support that greatly enriched this research project.

REFERENCES

- [1] Munir Ahmad, Muhammad Abdul Qadir, Atta Rahman, Rachid Zagrouba, Fahd Alhaidari, Tariq Ali, and Farzana Zahid. 2020. Enhanced query processing over semantic cache for cloud based relational databases. *Journal of Ambient Intelligence and Humanized Computing* (2020), 1–19.
- [2] Ouafa Ajarroud and Ahmed Zellou. 2022. SBQP: Towards a Semantic-Based Query Processing for Efficient Mediation Caching. In *Advances in Systems Engineering: Proceedings of the 28th International Conference on Systems Engineering, ICSEng 2021, December 14–16, Wroclaw, Poland* 28. Springer, 477–487.
- [3] Marwah Alaofi, Luke Gallagher, Mark Sanderson, Falk Scholer, and Paul Thomas. 2023. Can Generative LLMs Create Query Variants for Test Collections? An Exploratory Study. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1869–1873.
- [4] Juan C Alonso, Alberto Martin-Lopez, Sergio Segura, Jose Maria Garcia, and Antonio Ruiz-Cortes. 2022. ARTE: Automated Generation of Realistic Test Inputs for Web APIs. *IEEE Transactions on Software Engineering* 49, 1 (2022), 348–363.
- [5] Fu Bang. 2023. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. In *3rd Workshop for Natural Language Processing Open Source Software*.
- [6] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A Smith, and Matt Gardner. 2021. A dataset of information-seeking questions and answers anchored in research papers. *arXiv preprint arXiv:2105.03011* (2021).
- [7] Ophir Frieder, Ida Mele, Cristina Ioana Muntean, Franco Maria Nardini, Raffaele Perego, and Nicola Tonellotto. 2022. Caching Historical Embeddings in Conversational Search. *ACM Transactions on the Web* (2022).
- [8] Parke Godfrey and Jarek Gryz. 1999. Answering queries by semantic caches. In *International Conference on Database and Expert Systems Applications*. Springer, 485–498.
- [9] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2312–2323.
- [10] Yang Li, Yu Wu, Jinyu Li, and Shujie Liu. 2023. Prompting large language models for zero-shot domain adaptation in speech recognition. *arXiv preprint arXiv:2306.16007* (2023).
- [11] Zhe Liu, Chunyang Chen, Junjie Wang, Xing Che, Yuekai Huang, Jun Hu, and Qing Wang. 2023. Fill in the Blank: Context-aware Automated Text Input Generation for Mobile GUI Testing. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 1355–1367. <https://doi.org/10.1109/ICSE48619.2023.00119>
- [12] Leonardo Mariani, Mauro Pezzè, Oliviero Riganelli, and Mauro Santoro. 2014. Link: exploiting the web of data to generate test inputs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. 373–384.
- [13] Jaime Salas and Aidan Hogan. 2022. Semantics and canonicalisation of SPARQL 1.1. *Semantic Web* 13, 5 (2022), 829–893.
- [14] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2023. Software testing with large language model: Survey, landscape, and vision. *arXiv preprint arXiv:2307.07221* (2023).
- [15] Tanapuch Wanwarang, Nataniel P Borges Jr, Leon Bettscheider, and Andreas Zeller. 2020. Testing apps with real-world inputs. In *Proceedings of the IEEE/ACM 1st International Conference on Automation of Software Test*. 1–10.
- [16] Chenyang Yang, Rishabh Rustogi, Rachel Brower-Sinning, Grace A Lewis, Christian Kästner, and Tongshuang Wu. 2023. Beyond Testers’ Biases: Guiding Model Testing with Knowledge Bases using LLMs. *arXiv preprint arXiv:2310.09668* (2023).
- [17] Shengcheng Yu, Chunrong Fang, Yuchen Ling, Chentian Wu, and Zhenyu Chen. 2023. LLM for Test Script Generation and Migration: Challenges, Capabilities, and Opportunities. *arXiv:2309.13574 [cs.SE]*